

Cahier des charges

Health Quality Analytics

1. Contexte du projet

Dans le cadre de la transformation numérique du secteur de la santé, les établissements hospitaliers ont besoin d'outils modernes pour suivre la qualité des soins, analyser les incidents, gérer les risques QHSE et améliorer la prise de décision.

Le projet **Health Quality Analytics** consiste à concevoir et développer une application web permettant aux responsables qualité, aux responsables QHSE et aux administrateurs hospitaliers de suivre les indicateurs qualité, gérer les incidents, analyser les risques et générer des rapports.

L'objectif est de proposer une solution sécurisée, moderne et professionnelle basée sur **Spring Boot** pour le backend et **React JS** pour le frontend.

Partie Frontend

Technologies utilisées

- React JS 19
- JavaScript ES6+
- HTML5
- CSS3
- Vite
- Axios
- React Router
- Chart.js ou Recharts
- Git / GitHub

Fonctionnalités attendues

Page d'accueil / Dashboard

Créer une interface principale contenant :

- nombre total d'incidents,
- nombre d'audits,
- taux de conformité,

- score qualité global,
- score risque global.

Navbar / Sidebar

Créer une navigation contenant :

- logo ou nom de l'application,
- Dashboard,
- Incidents,
- Audits,
- Départements,
- Rapports,
- Utilisateurs,
- Profil,
- Déconnexion.

Gestion des incidents

Créer des pages permettant de :

- afficher la liste des incidents,
- ajouter un incident,
- modifier un incident,
- supprimer un incident,
- consulter les détails d'un incident,
- filtrer les incidents par type, gravité, statut ou département.

Gestion des audits qualité

Créer des interfaces permettant de :

- créer un audit,
- modifier un audit,
- consulter la liste des audits,
- suivre les résultats d'audit,
- calculer le taux de conformité.

Gestion des départements

Créer une page permettant de gérer :

- service urgence,
- service chirurgie,
- service pharmacie,
- service laboratoire,
- service hospitalisation.

Tableaux de bord analytiques

Afficher des graphiques pour :

- évolution mensuelle des incidents,
- répartition des incidents par type,
- répartition des incidents par gravité,
- taux de conformité par département,
- score risque par service.

Gestion des rapports

Créer une page permettant de :

- consulter les rapports générés,
- télécharger un rapport PDF,
- télécharger un rapport Excel,
- filtrer les rapports par date ou département.

Authentification

Créer les pages suivantes :

- inscription,
- connexion,
- profil utilisateur,
- page accès refusé.

Interactions JavaScript attendues

Ajouter au moins :

- menu responsive,
- dark mode / light mode,
- validation des formulaires,
- affichage dynamique des messages d'erreur,
- recherche et filtrage,
- pagination côté interface,
- graphiques dynamiques.

Composants React attendus

- Header
- Sidebar
- Dashboard
- IncidentList
- IncidentForm
- AuditList

- AuditForm
- DepartmentList
- ReportList
- Login
- Register
- ProtectedRoute
- ChartCard
- StatCard
- Footer

Contraintes frontend

- Utiliser des composants réutilisables.
- Séparer le code en fichiers clairs.
- Utiliser Axios pour communiquer avec le backend.
- Utiliser React Router pour la navigation.
- Créer une interface responsive.
- Gérer le token JWT côté frontend.
- Protéger les routes selon le rôle utilisateur.

Partie Backend

Technologies utilisées

- Java 17 ou Java 21
- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- PostgreSQL ou MySQL
- Maven
- Spring Security
- JWT
- MapStruct
- Lombok
- JUnit
- Mockito
- Swagger / OpenAPI
- Docker
- Docker Compose
- Redis
- GitHub Actions

Entités principales

User

- id
- username
- email
- password
- role

Department

- id
- name
- description

Incident

- id
- title
- description
- type
- gravity
- status
- incidentDate
- department
- reportedBy

Audit

- id
- title
- auditDate
- score
- conformityRate
- observations
- department

CorrectiveAction

- id
- title
- description
- status
- deadline

- incident
- responsibleUser

QualityIndicator

- id
- name
- value
- targetValue
- period
- department

Report

- id
- title
- type
- createdAt
- fileUrl

Fonctionnalités backend

Gestion des utilisateurs

- créer un utilisateur,
- modifier un utilisateur,
- supprimer un utilisateur,
- lister les utilisateurs,
- consulter un utilisateur,
- gérer les rôles.

Gestion des départements

- ajouter un département,
- modifier un département,
- supprimer un département,
- lister les départements,
- consulter un département.

Gestion des incidents

- déclarer un incident,

- modifier un incident,
- supprimer un incident,
- lister les incidents,
- rechercher par type,
- rechercher par gravité,
- rechercher par statut,
- rechercher par département,
- calculer le nombre total d'incidents.

Gestion des audits

- créer un audit,
- modifier un audit,
- supprimer un audit,
- consulter les audits,
- calculer le taux de conformité,
- rechercher les audits par département.

Gestion des actions correctives

- créer une action corrective,
- affecter une action à un responsable,
- modifier le statut,
- suivre la date limite,
- consulter les actions en retard.

Gestion des indicateurs qualité

Calculer automatiquement :

- taux d'incidents,
- taux de conformité,
- score qualité,
- score risque,
- nombre d'incidents critiques,
- évolution mensuelle des incidents.

Exemple :

Score risque =
(0.4 × nombre d'incidents critiques) +
(0.3 × nombre de non-conformités) +
(0.3 × actions correctives en retard)

Sécurité

Authentification JWT

Fonctionnalités attendues :

- inscription utilisateur,
- connexion utilisateur,
- génération du token JWT,
- validation du token,
- expiration du token,
- protection des endpoints.

Rôles utilisateurs

ADMIN

Peut gérer :

- utilisateurs,
- départements,
- incidents,
- audits,
- rapports,
- indicateurs.

QUALITY_MANAGER

Peut :

- gérer les incidents,
- gérer les audits,
- consulter les dashboards,
- générer les rapports.

QHSE_MANAGER

Peut :

- gérer les risques,
- suivre les actions correctives,
- consulter les statistiques QHSE,
- générer les rapports.

STAFF

Peut :

- déclarer un incident,
- consulter ses incidents,
- suivre les actions qui lui sont affectées.

Concepts obligatoires

- AuthenticationManager
- PasswordEncoder
- SecurityFilterChain
- JWT Filter
- UserDetailsService
- BCryptPasswordEncoder
- Roles / Authorities

Pagination, recherche et tri

Ajouter la pagination pour :

- incidents,
- audits,
- utilisateurs,
- départements,
- actions correctives.

Ajouter le tri par :

- date d'incident,
- gravité,
- statut,
- département,
- date d'audit.

Ajouter la recherche par :

- type d'incident,
- gravité,
- statut,
- département,
- utilisateur responsable.

Reporting

L'application doit permettre de générer :

- rapport PDF des incidents,
- rapport PDF des audits,
- rapport qualité mensuel,
- rapport QHSE par département,
- export Excel des incidents.

Cache avec Redis

Utiliser Redis pour améliorer les performances des endpoints suivants :

- dashboard principal,
- liste des incidents,
- liste des audits,
- indicateurs qualité,
- statistiques par département.

Le cache doit être invalidé lors de :

- création,
- modification,
- suppression.

Tests

Créer des tests avec :

- JUnit,
- Mockito.

Tester :

- services,
- controllers,
- repositories,
- sécurité,
- calcul des scores qualité et risque.

Qualité du code

Utiliser :

- SonarQube ou Qodana,
- architecture MVC,
- DTO,
- Mapper,
- gestion globale des exceptions,
- validation des données,
- commentaires utiles,
- code propre et structuré.

Docker et déploiement

L'application doit contenir :

- Dockerfile backend,
- Dockerfile frontend,
- docker-compose.yml,
- conteneur base de données,
- conteneur Redis,
- conteneur backend,
- conteneur frontend.

CI/CD GitHub Actions

Créer un pipeline permettant de :

- compiler le backend,
- lancer les tests,
- vérifier le build,
- construire les images Docker,
- préparer le déploiement.

Le workflow doit s'exécuter à chaque :

- push
- pull request.

Déploiement Cloud

Déployer l'application sur :

- Render
- Railway

L'application doit être accessible via une URL publique.

Conception UML

Créer obligatoirement :

- diagramme de classes,
- diagramme de cas d'utilisation,
- diagramme de séquence.

Exemple de séquence :

Déclaration d'un incident → validation → enregistrement → calcul du score risque → mise à jour du dashboard → notification.

Livrables

Support de présentation

Fournir un support de présentation au format :

- PDF
- PowerPoint (PPT)
- Canva

Le support devra présenter :

- le contexte du projet,
- les objectifs,
- les fonctionnalités réalisées,
- l'architecture technique,
- les technologies utilisées,
- les diagrammes UML,
- les captures d'écran de l'application,
- les résultats obtenus,
- les perspectives d'évolution.

Planification des tâches

Fournir un lien vers un espace de gestion de projet :

- Jira
- Trello

L'outil devra contenir :

- le backlog du projet,
- les user stories,
- les tâches réalisées,
- les statuts d'avancement,
- l'organisation du projet.

Design final

Fournir un lien vers la maquette finale réalisée avec :

- Figma
- Adobe XD
- ou tout autre outil équivalent

La maquette devra inclure :

- les écrans principaux,
- la navigation utilisateur,
- les interfaces responsives,
- les composants graphiques utilisés.

Repository GitHub

Fournir un lien vers le dépôt GitHub contenant l'ensemble des éléments du projet.

Le repository devra contenir :

README

Une documentation complète présentant :

- la description du projet,
- les objectifs,
- les technologies utilisées,
- l'architecture globale,
- les instructions d'installation,
- les instructions d'exécution,
- les captures d'écran principales,
- les informations de déploiement.

Diagrammes UML

Les diagrammes suivants au format image (PNG, JPG ou JPEG) :

- Diagramme de classes
- Diagramme de cas d'utilisation (Use Case)
- Diagramme de séquence

Code source

L'ensemble du code source nécessaire au fonctionnement du projet :

- Frontend React JS
- Backend Spring Boot
- Configuration Spring Security JWT
- Configuration Docker
- Configuration Docker Compose
- Configuration Redis
- Configuration GitHub Actions
- Scripts SQL éventuels

Documentation API

- Collection Postman
- Documentation Swagger / OpenAPI

Pipeline CI/CD

Configuration GitHub Actions permettant :

- le build automatique,
- l'exécution des tests,
- la validation du projet avant déploiement.

Déploiement

Lien public vers l'application déployée :

- Frontend
- Backend API
- Documentation Swagger

Application fonctionnelle

Une version complète, testée, sécurisée et opérationnelle de l'application **Health Quality Analytics**, démontrant l'ensemble des fonctionnalités décrites dans le cahier des charges.

Résultat attendu

À la fin du projet, l'application **Health Quality Analytics** doit permettre à un établissement de santé de suivre la qualité des soins, analyser les incidents, gérer les risques QHSE, calculer des indicateurs qualité et produire des rapports décisionnels.